```
import os
import time
import json
import uuid
import logging
import shutil
from contextlib import asynccontextmanager
from datetime import datetime, timezone
from fastapi import FastAPI, HTTPException, Request, UploadFile, File
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.responses import Response
from pydantic import BaseModel
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded
from dotenv import load_dotenv
from langchain_core.callbacks import BaseCallbackHandler

from rag import get_rag_chain
from ingest import load_docs, chunk_documents, store_chunks

load_dotenv()


logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

rag_chain = None
limiter = Limiter(key_func=get_remote_address)
```

```
UPLOAD_DIR = "uploaded_docs"
ALLOWED_EXTENSIONS = {".pdf", ".txt"}

API_TRACE_LOG = os.environ.get("API_TRACE_LOG", "api_traces.jsonl")
CHAT_TRACE_LOG = os.environ.get("CHAT_TRACE_LOG", "chat_traces.jsonl")
UPLOAD_TRACE_LOG = os.environ.get("UPLOAD_TRACE_LOG",
"upload_traces.jsonl")



def append_trace(log_path: str, record: dict):
    with open(log_path, "a") as f:
        f.write(json.dumps(record) + "\n")



class RequestTracingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        request_id = str(uuid.uuid4())
        request.state.request_id = request_id
        start = time.perf_counter()


        try:
            response = await call_next(request)
        except Exception:
            latency_ms = round((time.perf_counter() - start) * 1000, 2)
            append_trace(API_TRACE_LOG, {
                "request_id": request_id,
                "timestamp": datetime.now(timezone.utc).isoformat(),
                "method": request.method,
```

```
                "path": request.url.path,
                "status_code": 500,
                "latency_ms": latency_ms,
            })
            logger.exception(f"[{request_id}] unhandled error on {request.url.path}")
            raise

        latency_ms = round((time.perf_counter() - start) * 1000, 2)
        response.headers["X-Request-ID"] = request_id

        append_trace(API_TRACE_LOG, {
            "request_id": request_id,
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "method": request.method,
            "path": request.url.path,
            "status_code": response.status_code,
            "latency_ms": latency_ms,
        })
        logger.info(f"[{request_id}] {request.method} {request.url.path} "
                    f"-> {response.status_code} ({latency_ms}ms)")
        return response

class StageTimingCallback(BaseCallbackHandler):
    def __init__(self):
        self.timings = {}
        self._retriever_start = None
        self._llm_start = None

    def on_retriever_start(self, serialized, query, **kwargs):
```

```
        self._retriever_start = time.perf_counter()

    def on_retriever_end(self, documents, **kwargs):
        if self._retriever_start is not None:
            self.timings["retrieval_ms"] = round((time.perf_counter() - self._retriever_start) *
1000, 2)

    def on_chat_model_start(self, serialized, messages, **kwargs):
        self._llm_start = time.perf_counter()

    def on_llm_start(self, serialized, prompts, **kwargs):
        self._llm_start = time.perf_counter()

    def on_llm_end(self, response, **kwargs):
        if self._llm_start is not None:
            self.timings["generation_ms"] = round((time.perf_counter() - self._llm_start) *
1000, 2)



def score_groundedness(answer: str, context_docs: list) -> float:
    context_words = set()
    for d in context_docs:
        context_words.update(d.page_content.lower().split())
    answer_words = set(answer.lower().split())
    if not answer_words:
        return 0.0
    return round(len(answer_words & context_words) / len(answer_words), 3)

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
```

```
rag_chain =None
limiter = Limiter(key_func=get_remote_address)

UPLOAD_DIR = "uploaded_docs"
ALLOWED_EXTENSIONS = {".pdf", ".txt"}

@asynccontextmanager
async def lifespan(app:FastAPI):
    global rag_chain
    logger.info("building RAG chain ...")
    rag_chain = get_rag_chain()
    app.state.rag_chain = rag_chain
    os.makedirs(UPLOAD_DIR, exist_ok=True)
    logger.info("RAG chain is ready.")

    yield {"rag_chain": rag_chain}

    logger.info("shutting down.")

app = FastAPI(title="RAG api", version="1.0.0", lifespan=lifespan)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)
app.add_middleware(RequestTracingMiddleware)


class Queryrequest(BaseModel):
    question: str
    k: int | None = None
```

```
class Sourcechunk(BaseModel):
    content: str
    metadata: dict

class Queryresponse(BaseModel):
    answer: str
    sources: list[Sourcechunk]

class UploadResponse(BaseModel):
    filename: str
    chunks_stored: int
    message: str

@app.get("/")
def endpoint():
    return {"message":"your chatbot is ready for chat."}

@app.get("/status")
def health():
    return {
            "status": "unhealthy" if rag_chain is None else "healthy",
            "model": rag_chain is not None
        }

@app.post("/upload",response_model=UploadResponse)
@limiter.limit("5/minute")
async def upload_document(request: Request, file: UploadFile = File(...)):
    request_id = request.state.request_id
```

```
trace = {"request_id": request_id, "filename": file.filename, "timings_ms": {}}

ext = os.path.splitext(file.filename)[1].lower()
if ext not in ALLOWED_EXTENSIONS:
    raise HTTPException(
        status_code=400,
        detail=f"Unsupported file type '{ext}'. Allowed: {ALLOWED_EXTENSIONS}",
    )

save_path = os.path.join(UPLOAD_DIR, file.filename)
t0 = time.perf_counter()
try:
    with open(save_path, "wb") as f:
        shutil.copyfileobj(file.file, f)
except Exception as e:
    logger.exception(f"[{request_id}] failed to save uploaded file")
    raise HTTPException(status_code=500, detail=f"Could not save file: {e}")
finally:
    file.file.close()
trace["timings_ms"]["save"] = round((time.perf_counter() - t0) * 1000, 2)

try:
    t0 = time.perf_counter()
    documents = load_docs(save_path)
    trace["timings_ms"]["load"] = round((time.perf_counter() - t0) * 1000, 2)
    trace["num_documents"] = len(documents)

    t0 = time.perf_counter()
    chunks = chunk_documents(documents)
```

```
        trace["timings_ms"]["chunk"] = round((time.perf_counter() - t0) * 1000, 2)
        trace["num_chunks"] = len(chunks)

        t0 = time.perf_counter()
        store_chunks(chunks)
        trace["timings_ms"]["store"] = round((time.perf_counter() - t0) * 1000, 2)
    except Exception as e:
        trace["error"] = str(e)
        append_trace(UPLOAD_TRACE_LOG, trace)
        logger.exception(f"[{request_id}] ingestion failed")
        raise HTTPException(status_code=500, detail=f"Ingestion failed: {e}")
    finally:
        if os.path.exists(save_path):
            os.remove(save_path)

    trace["success"] = True
    append_trace(UPLOAD_TRACE_LOG, trace)

    return UploadResponse(
        filename=file.filename,
        chunks_stored=len(chunks),
        message="File ingested and stored successfully.",
    )

@app.post("/chat",response_model=Queryresponse)
@limiter.limit("10/minute")
async def query(request: Request, req: Queryrequest):
    if not req.question or not req.question.strip():
        raise HTTPException(status_code=400, detail="Question cannot be empty")
```

```
request_id = request.state.request_id
callback = StageTimingCallback()
t0 = time.perf_counter()

try:
    results = rag_chain.invoke({"input": req.question}, config={"callbacks": [callback]})
except Exception as e:
    append_trace(CHAT_TRACE_LOG, {
        "request_id": request_id,
        "question": req.question,
        "error": str(e),
        "timings_ms": callback.timings,
    })
    logger.exception(f"[{request_id}] RAG chain failed.")
    raise HTTPException(status_code=500, detail=str(e))

total_ms = round((time.perf_counter() - t0) * 1000, 2)
context_docs = results.get("context", [])
answer = results["answer"]

sources = [
    Sourcechunk(content=doc.page_content[:300], metadata=doc.metadata)
    for doc in context_docs
]

append_trace(CHAT_TRACE_LOG, {
    "request_id": request_id,
    "timestamp": datetime.now(timezone.utc).isoformat(),
```

```
    "question": req.question,
    "answer": answer,
    "num_sources": len(sources),
    "sources": [{"source": s.metadata.get("source", "unknown")} for s in sources],
    "groundedness": score_groundedness(answer, context_docs),
    "timings_ms": {**callback.timings, "total_ms": total_ms},
})

return Queryresponse(answer=answer, sources=sources)
```